

BASALT

A Property- and Behavior-Aware Mathematical Language over Lean

Refinement-directed elaboration, relational contracts,
scoped reasoning, and certified symbolic computation

Central proposal. Let a mathematician describe not only what kind of object a term is, but which properties it has, which relations its maps preserve, and what an operation promises to do. Infer the routine evidence needed to use those descriptions, while emitting ordinary Lean terms and retaining the exact statement, scope, and computational guarantee.

Prepared for Vladimir Reshetnikov

6 September 2026

Evidence boundary. Both round-two synthesis sources were read closely, and selected Lean and Haskell modules were inspected directly. This article specifies a language; it does not claim an implemented compiler or measured usability improvement. The accompanying Python demonstrations were run. The illustrative Lean core was not compiled in this environment.

Abstract

A mathematician rarely thinks of a function merely as a term of type $A \rightarrow B$. It may be an increasing function on an interval, a measure-preserving map, an ideal-preserving algebra homomorphism, or a procedure that returns all solutions rather than one. Lean can already express these distinctions, but ordinary authoring often separates the object from the proofs that make its intended use legitimate. The resulting effort appears as repeated side-condition assembly, coercion alignment, local-to-global bookkeeping, and transport between representations.

BASALT proposes a conservative extension of the *elaborated type language*: object-indexed refinements, dependent behavioral contracts, and multi-object relational contracts become first-class guides to evidence synthesis. The kernel’s logic and definitional equality remain unchanged. A local refinement enriches knowledge about the same term; an exported refinement becomes a subtype or a record with proof fields. A relation such as exactness belongs to an entire diagram, not to an adjective on each vertex. Locality and precision are explicit indices of evidence, not informal levels of confidence. An untrusted planner selects theorem applications, synthesis providers, and certified computations; acceptance requires the exact requested Lean proposition.

The proposal develops two substantial algebraic cases from the FLT repository, a measure-theoretic case from ProveIt, and a locality control from its autonomous ODE development. It also examines Leant’s finite-spine length-contract handoff, which distinguishes candidate provenance from behavioral proof. A practical multivariate polynomial checker is specified down to coefficient denotation, normalization, and original-target reconstruction. The article supplies typing rules, composition theorems, failure examples, a conservative implementation path, a fair evaluation design, and a source register. The proposed advance over round two is not another name for a certified answer: it is a type-directed account of how mathematical knowledge is accumulated and composed before an answer is requested.

Contents

1	The thesis: make mathematical knowledge part of the interface	3
2	What the second round establishes, and what remains open	4
3	Fresh source diagnosis: four different kinds of verbosity	5
4	The surface language and its three views	6
5	Extending the type system without extending the kernel	7
6	A small refinement-and-evidence calculus	9
7	Behavioral types: functions with mathematical obligations	11
8	Relational and scoped types: what belongs to the whole situation	13
9	Evidence synthesis: demand-driven, bounded, and subject-preserving	14
10	Worked case I: probability, atomlessness, and CDF symmetry	16
11	Worked case II: ideal-preserving maps and completion functoriality	17

12 Worked case III: exactness is a relational type	19
13 Control case: range-local analytic induction	21
14 Leant integration: from structural candidates to proved behavior	22
15 Certified CAS: a concrete multivariate checker, not a slogan	24
16 Observation-indexed contracts connect jets and quotient towers	27
17 Definitions, choices, and the boundary between mathematics and execution	28
18 Three correctness boundaries, not one green checkmark	29
19 Architecture and implementation plan	30
20 Relation to prior type-system and proof-language work	32
21 Evaluation: make the new layer earn its place	33
22 Risks, unresolved questions, and the recommended next artifact	35
23 Conclusion	35
A Answers to the third-round questions in the two syntheses	36
B A minimal ordinary Lean core, with its validation status	37
C Source register and scope of inspection	37

1 The thesis: make mathematical knowledge part of the interface

The right target is not the shortest possible proof script. It is the shortest *faithful argument*: one that preserves the intended objects, assumptions, quantifiers, and reasons while delegating details that can be reconstructed reliably. A one-line invocation of a theorem identical to the goal is short, but it does not explain a mathematical argument. Conversely, a long reusable lemma may be the right place to pay for a difficult transport theorem once.

I propose BASALT, a mathematical authoring layer whose fundamental judgment is not simply “this expression has this type,” but:

this exact object has this type and these proved properties, in this scope.

For maps and algorithms, the properties include *behavior*: inputs meeting specified conditions yield outputs satisfying specified relations, or related inputs yield related outputs. For diagrams, properties can involve several objects and arrows simultaneously. All these assertions elaborate to ordinary Lean propositions and proofs.

The author should be able to write, schematically,

```
let mu be an atomless probability measure on Real
let F := cdf mu
assume mu is invariant under (x |-> a - x)

claim F(a - x) = 1 - F(x)
  by pushforward, complement, and null-boundary replacement
```

The three named steps determine a calculational plan. The system retrieves the probability and atomlessness evidence for this particular μ , generates the relevant measurable-set obligations, and checks the endpoint conversion. It must not replace the plan with an unrelated theorem while continuing to claim that those steps were verified. Nor may it silently add atomlessness when the author supplied only a probability measure.

The design is intentionally hybrid. It combines a mathematical document, a richer elaboration discipline, theorem-directed proof synthesis, and a certificate-oriented CAS. None of these components gains authority from its name. A mathematical property must have a proof, a computation must have its advertised guarantee, and a suggestion must survive insertion at its original target. This inherits the strongest boundaries of both second-round syntheses [1, 2].

1.1 What is genuinely being extended

Lean already has dependent functions, subtypes, structures with proof fields, and propositions expressing arbitrarily rich mathematics. The proposed extension is therefore *not additional logical expressiveness*. It is a new authoring and elaboration discipline that makes those existing facilities work together with fewer explicit transports and less repeated hypothesis assembly. In particular, it makes a distinction between discovering a proof of a property, recording that property on a stable subject, and later using it as a typing prerequisite.

An ordinary type can indeed be a behavioral specification when it is sufficiently dependent or refined. The slogan “a type is not a behavioral specification” should be read as a warning about a *coarse* type such as $\text{List } A \rightarrow \text{List } A$, not as a limitation of dependent type theory. A sorting contract, for example, can be a subtype of functions equipped with a universal proof of sortedness and permutation preservation.

1.2 Five commitments that distinguish this proposal

The specific proposal consists of five interacting choices. First, locally learning a property does not construct a new mathematical object. Second, function contracts and diagram contracts are handled alongside unary refinements. Third, evidence synthesis is driven by the requirements of the current operation rather than by an attempt to infer every true property. Fourth, locality, approximation, and representation change require typed transport theorems. Fifth, abstract behavioral analysis is useful for pruning and composition, but it becomes a Lean theorem only through proved primitive laws and an interpretation theorem.

These are design choices to test, not claims of historical invention. Their combination is developed against source examples in Sections 10–14, rather than justified by aesthetic preference for the sample syntax.

2 What the second round establishes, and what remains open

The two requested reports are *Mathematical Objects, Certified Computation* and *Nine Workbenches*. Both were inspected as TeX sources, including their corrections, evaluation proposals, and questions for a further round. Their current versions explicitly discuss each other’s revisions; they should not be treated as untouched independent votes [1, 2].

The first organizes agreement into eight broad commitments. The second gives a finer editorial matrix and a consolidated mutation catalogue. The counts have different denominators and do not establish either novelty or usability. Both reports converge on an important acceptance boundary: a producer returns data and evidence for the caller’s *fixed specification*. A conditional identity, a finite jet, and a complete solution predicate are different mathematical answers. A presentation remains connected to its original semantic object.

Round-two principle	Retained in BASALT	Third-round extension
Fixed object and checked relation	No silent change of anchor or guarantee	Local refinements accumulate without replacing the anchor
Dependent guards	Conditions remain in their witness scope	Behavioral contracts derive operation-specific prerequisites
Annotated theorem interfaces	Ordinary Lean theorems are logical contracts	Refinement and relation indices select theorem roles
Exact-target providers	Acceptance is not a suggestive success label	Abstract behavioral laws must reconnect to exact Lean providers
Representation discipline	Preservation and reflection are separate	Commuting diagrams and observation consumers carry relational contracts
Retained evidence	Publication does not require rediscovery	Property dependencies support targeted invalidation and semantic merging
Fair Lean controls	Libraries and services are shared	Measure the incremental value of refinement-directed elaboration

2.1 Corrections that must survive the next design

Several repairs in the reports are especially instructive. A candidate solution set needs both soundness and coverage; one witness plus coverage is insufficient. Equality at a point does not become neighborhood equality merely because the ambient domain is open. An enclosure can prove equality when its bounds collapse, so a blanket prohibition would be too restrictive. A

nonzero element need not be a unit, and even a cancellable element need not be invertible. Uniform sequence identities cannot be shortened by moving a quantifier inside an equivalence [1, 2].

These are not only checker problems. They are mistakes in choosing the *type of the requested evidence*. A stronger type discipline is valuable precisely when it prevents these invalid promotions before search begins, and explains which premise is missing when a proposed bridge does not apply.

2.2 What not to repeat

The earlier proposals already give elaborate versions of a dependent pair (y, p) with $p : \text{Spec}(x, y)$. Repeating that interface under a tenth name would add little. They also discuss the same binomial, autonomous-derivative, and formal-series examples repeatedly. This article retains the autonomous example as a control, but develops measure-theoretic and algebraic examples with different proof obligations. Once inspected, these are *development cases*, not held-out evaluation tasks.

Nor does this article reinterpret the reports’ executed experiments as its own. Their Lean probes and polynomial timings are historical evidence. No aggregate ProveIt or FLT build was attempted here. The distinction between a proved Boolean checker outcome and the unproved denotational soundness of that checker is addressed explicitly in Section 15.

3 Fresh source diagnosis: four different kinds of verbosity

The source register in Appendix C identifies the inspected revisions and files. This is a focused design study, not a repository-wide quality audit. The FLT repository’s README reports extensive independent checking; that report is not a checking result reproduced here. Its warning that a declaration’s statement, rather than its generated name or English summary, determines what was proved is directly relevant to this project [5].

3.1 Repeated property assembly in measure theory

ProveIt’s `ContinuousCDF.lean` develops continuity of an atomless probability CDF, reflection symmetry, and identification of a measure from an everywhere derivative of its CDF. Its reflection calculation moves through pushforward, preimage, complement, and an almost-everywhere equality of open and closed half-lines. The type `Measure ℝ` alone does not tell the elaborator which of these steps is available; the probability and null-singleton hypotheses do [3].

The opportunity is not to hide those hypotheses. It is to attach their evidence to μ , derive the precise required consequences on demand, and retain the reason for the endpoint step. In particular, the null-singleton property is indexed by the measure: atomlessness of a different measure is irrelevant.

3.2 Scope and locality in analytic induction

The autonomous-derivative module proves an all-orders statement by induction. The induction hypothesis holds on an open set, so at each point it supplies eventual equality in the neighborhood filter. Only then may the proof replace a derivative. The derivative assumptions on auxiliary functions are needed only on the range of the original solution, not everywhere [4].

This is not just positivity-style automation. It is a change in the kind and scope of evidence:

a universally quantified equality on an open set is converted into local equality at a particular point. A property language that records only “the expressions are equal” would erase the crucial distinction.

3.3 Property-preserving maps in completions

The FLT module `Def_AdicCompletionRingFunctoriality.lean` starts with an algebra homomorphism $f : R \rightarrow S$ and ideals I, J , together with $f(I) \subseteq J$. It builds maps on all quotients by powers of the ideals, proves compatibility with transition maps, and lifts to completions. Many lemmas repeatedly carry ideal-preservation evidence and relate the same map through several bundled representations [6].

Here the useful enriched type is not merely “homomorphism.” It is a morphism between *rings with selected ideals*, preserving the selected ideals. Composition can then construct the preservation proof as part of its result. The mathematical map remains $g \circ f$; the additional type information explains when quotient and completion operations are legal.

3.4 Relations on diagrams in exactness arguments

The proof of `S_Rep_indBotSC_shortExact.lean` constructs a linear model of an induced representation, proves that the modeling equivalences commute with maps, tensors a short exact sequence with a flat module, and transports the result back. Its final theorem is already short; the infrastructure establishing the model and the commutative squares contains the substantive work [7].

Exactness is a property of a pair of composable arrows, and transport requires a compatible ladder of equivalences. Attaching independent labels such as “injective” and “surjective” to isolated arrows does not capture all of this. A language that only adds unary refinements will miss one of the main sources of algebraic proof structure.

Design decision. Treat unary properties, behavioral contracts, and diagram relations as different surface forms of proved predicates, with different indexing and elaboration policies. Do not force all mathematical knowledge into a hierarchy of adjective-bearing object classes.

4 The surface language and its three views

BASALT has an authoring view, an expanded mathematical view, and an ordinary Lean export. They are views of the same accepted evidence, not separate proofs that are expected to agree by convention. All syntax in this article is proposed unless explicitly identified as Lean.

4.1 A small vocabulary

The initial frontend needs a modest vocabulary: introduce objects; assert properties; define a map by a contract; make a claim with a reason; calculate; change representation; choose a witness satisfying a specification; and inspect an unresolved obligation. Existing Lean terms remain usable inside these forms.

```

context k : commutative ring
context (R, I), (S, J), (T, K) : k-algebras with ideals

let f : (R, I) => (S, J) preserving ideals
let g : (S, J) => (T, K) preserving ideals
let h := g after f

claim completion(h) = completion(g) after completion(f)
  by equality at every quotient level

```

The notation for pairs selects the actual algebra structures and ideals. It does not ask typeclass search to invent an ideal, a scalar action, or a ring structure. The final reason creates one schematic quotient-level equality, not a finite test of several quotient levels.

4.2 What appears in the expanded view

For the preceding example, the expanded view displays f, g , the statements $f(I) \subseteq J$ and $g(J) \subseteq K$, the derived statement $(g \circ f)(I) \subseteq K$, the completion-map theorem used, and its all-levels extensionality premise. Routine applications of membership lemmas may be folded away. The choice of the scalar ring and the direction of the maps may not.

A concise display is not necessarily a single line. If a solver returns a conditional result, its condition belongs next to the result. If it returns a finite observation, its precision belongs next to that observation. An unresolved condition is not a cosmetic warning whose acknowledgment makes it true.

4.3 When ordinary Lean should remain the surface

Short existing proofs should remain short. The final induction-of-exactness corollary in the inspected FLT module already consists largely of applying its prepared bridge. Translating it into a longer narrative would be a regression. The first product should support lifting an existing Lean statement into a property-aware display and selectively introducing new forms, not demand conversion of an entire development.

5 Extending the type system without extending the kernel

5.1 What Lean already supplies

Lean’s subtypes represent values satisfying predicates, with a value field and a proposition-valued proof field. They can be used like the underlying value through a coercion, and their proof fields are computationally irrelevant [12]. Bundled maps and ordinary structures already express many mathematical interfaces. BASALT should reuse these mechanisms rather than replace them with a parallel mathematical universe.

The difficult part is systematic inference and use of those properties. An author knows that the current measure is atomless; the next lemma expects an almost-everywhere equality of two sets under that measure. A synthesis procedure needs a route between these facts, must instantiate it at the right subject, and should explain the route if it affects the argument.

5.2 Three possible extension levels

Library only. New bundled structures, theorem wrappers, and tactics can expose the desired properties without changing syntax. This is the necessary baseline and may be the best eventual product.

Elaboration-level refinements. A frontend accepts refinements, behavioral arrows, and relational declarations, then emits ordinary Lean terms with explicit proof arguments. This is the recommended experiment. It changes the programmer-facing type system while preserving the foundational one.

Kernel-level refinements or conversion. A primitive refinement constructor could be given a conservative interpretation, but it would add a new trusted implementation and migration surface for a feature already representable by existing types. A more ambitious rule making theorem-level property implication part of conversion would couple type checking to arbitrary proof search. Neither change is justified as an initial step.

Design decision. Implement an extended elaboration calculus first. Keep semantic subtyping proof-producing and explicit in the elaborated output. Do not ask the kernel to decide arbitrary predicate implication or to run a CAS as part of definitional equality.

5.3 Local refinements versus exported values

There are two distinct operations that should look related without being confused.

Local enrichment retains $x : A$ and a proof $p : P(x)$ in the existing context. A property index records where that proof can be found. A later proof $q : Q(x)$ adds knowledge about the same x . No rebinding to a new subtype value is required.

Packaging constructs a first-class value of

$$\text{Ref}(A, P) := \{x : A \mid P(x)\}.$$

It is useful at module boundaries, in containers of certified objects, or when the property is part of an API. It elaborates to a subtype or an ordinary record. The two modes are linked by introduction and projection, but the compiler must not introduce unnecessary wrappers after every local deduction.

For a row of properties $\mathcal{P} = (P_1, \dots, P_r)$ on a fixed base type,

$$\text{Ref}(A, \mathcal{P}) = \{x : A \mid P_1(x) \wedge \dots \wedge P_r(x)\}.$$

This is an intersection of refinements of the *same* object, not a product of unrelated objects. A property such as preservation of a selected ideal may mention parameters already in the context.

5.4 Properties are not structure selection

“ R is a field” can mean either that a chosen ring structure has additional laws or that one is introducing new data for a field structure. These are not interchangeable. Multiple topologies, norms, scalar actions, or bases on the same carrier can change a statement’s meaning. A refinement may add a proof about a selected structure; it must not silently select another structure.

The frontend therefore separates **withstructure** from **satisfyingproperty**. Property closure can use a theorem about the current topology. Changing the topology is a data-level operation with its own visible transport. Likewise, a proof that an element is nonzero is not permission to replace a commutative ring by a field.

5.5 Why not turn every property into a typeclass?

A small collection of familiar proposition-valued classes is useful for interoperability. Nevertheless, turning every derived proposition into a global instance would blur lexical scope, make search behavior difficult to predict, and invite cycles between equivalent descriptions. A relational fact indexed by a particular diagram is especially ill-suited to global instance search.

The proposed fact index is local, proof-bearing, and demand-driven. It may install a local instance when a known Lean API requires one, but it retains the proof’s exact source and dependencies. Typeclass resolution for mathematical structures and theorem search for property consequences remain distinct operations with separately bounded policies.

6 A small refinement-and-evidence calculus

This section specifies enough of the translation to make “conservative” a checkable design claim. It is a metatheoretic design, not a completed formalization of an elaborator.

6.1 Contexts and judgments

Let $\mathcal{E}$ be a Lean environment and Γ an ordered dependent context. Every accepted entry in a fact index has the form

$$(\text{subject tuple } \vec{t}, \varphi, p), \quad \mathcal{E}; \Gamma \vdash p : \varphi(\vec{t}).$$

The fact index stores handles and provenance; it does not enlarge the logic. Its entries must already be derivable in the Lean context, or be explicit hypotheses in that context. A displayed subject name is not its identity.

Write $e \rightsquigarrow t : A$ for elaboration of a source expression to a Lean term, and $e \Leftarrow T \rightsquigarrow t$ for checking against an expected enriched type. During elaboration, obligations may be unresolved; then the result is a *pending derivation*, not an accepted term. Publication requires all proof holes and data holes to be resolved under the chosen axiom policy.

The initial enriched type grammar is deliberately small:

$$T ::= A \mid \{x : A \mid \varphi(x)\} \mid (x : T) \rightarrow T' \mid \text{an ordinary dependent record.}$$

Behavioral and diagram contracts elaborate into instances of this grammar. The mathematical predicate language is Lean’s proposition language, not a second restricted logic pretending to express every theorem.

6.2 Refinement introduction and elimination

The introduction rule is

$$\frac{\Gamma \vdash e \rightsquigarrow t : A \quad \Gamma \vdash p : \varphi(t)}{\Gamma \vdash e \Leftarrow \{x : A \mid \varphi(x)\} \rightsquigarrow \langle t, p \rangle}.$$

Its premises show the separation of term elaboration from evidence synthesis. An obligation solver can propose p , but it cannot change the already fixed meaning of t to make $\varphi(t)$ easier.

Elimination projects the value and makes its proof available:

$$z : \{x : A \mid \varphi(x)\} \rightsquigarrow z.\text{val} : A, \quad z.\text{property} : \varphi(z.\text{val}).$$

A local enrichment is just a local proof declaration about t . A row projection uses conjunction elimination; adding a property uses conjunction introduction. None of these rules needs a new axiom.

6.3 Subtyping means a checked adapter

Given a proof

$$h : \forall x : A, P(x) \rightarrow Q(x),$$

there is a value-preserving adapter

$$\text{weaken}_h : \text{Ref}(A, P) \rightarrow \text{Ref}(A, Q), \quad \langle x, p \rangle \mapsto \langle x, h(x, p) \rangle.$$

The elaborator may insert it automatically at a checking boundary. The insertion is not a new definitional equality between the two subtype types. Even when $P \leftrightarrow Q$, a propositional equivalence is translated into maps or an explicit transport, not silently installed as kernel conversion.

Proof irrelevance helps with different proofs of the *same* proposition. It does not identify different values, different predicates, different chosen structures, or incompatible relation instances. The data identity must remain visible in every adapter.

6.4 Dependent application and unresolved guards

Suppose $f : (x : \text{Ref}(A, P)) \rightarrow B(x.\text{val})$ and $a : A$. Applying f to a generates the obligation $P(a)$ and elaborates, once that obligation is proved, to $f\langle a, p \rangle$. If $P(a)$ remains open, the application is pending. A frontend may offer a conditional theorem instead, but only as a different result whose condition is shown.

When outputs or later premises depend on previous witnesses, the order of the telescope is retained. The forms

$$\forall h : G, \{y : B \mid R(h, y)\} \quad \text{and} \quad \{y : B \mid \forall h : G, R(h, y)\}$$

are different interfaces. No binder may be moved simply because its name is hidden in the compact display. If the type of y itself depends on h , even the second expression needs a different formulation.

6.5 Relative soundness of the core rules

Proposition 6.1 (Conservative translation of the displayed core). *Assume the base expressions and registered theorem constants are well typed in $\mathcal{E}$, and every proof premise supplied to the displayed rules is checked by Lean in its indicated context. A finite accepted derivation using base elaboration, refinement introduction and projection, conjunction rules, checked weakening, dependent abstraction and application, and explicit equality transport translates to a well-typed term of its interpreted type in Lean.*

Proof. Induct on the derivation. Base cases are the assumed Lean judgments. Refinement introduction is the subtype constructor with its two checked arguments; projection is a structure projection. Row rules are constructors or projections of conjunctions. Weakening applies the supplied implication to the value and its existing proof. Abstraction and application are the corresponding dependent function rules. Equality transport applies Lean’s ordinary equality eliminator. The induction hypothesis supplies each subderivation at the exact context required by the next rule. $\square$

This proposition is deliberately modest. It explains why the suggested rules need no new logic. It does not verify a parser, a mutable elaboration engine, a serialization format, the property index, or the default display. Those components need implementation tests and, where warranted, formal verification. It also gives no completeness theorem for automatic evidence search.

7 Behavioral types: functions with mathematical obligations

A useful function interface says more than which objects can be passed to it. It can state which preconditions suffice, what is guaranteed about the output, and which relations the function preserves. These are properties of the *selected function*, not facts guessed from its name.

7.1 A total function with a conditional contract

For $f : A \rightarrow B$, a deterministic input–output contract is

$$\text{Contract}(f, P, Q) := \forall x : A, P(x) \rightarrow Q(x, f(x)).$$

The predicate Q can refer to both input and output. A packaged function is

$$\{f : A \rightarrow B \mid \text{Contract}(f, P, Q)\}.$$

Here f remains a total Lean function; the contract promises the specified behavior only under P . This is different from a function whose domain is itself $\text{Ref}(A, P)$, and different again from a partial computation returning an optional result. The surface may support all three, but its default must preserve the meaning of an existing Lean declaration.

```
construct transform : List A -> List A
  ensures (xs, ys) => ys is a permutation of xs

construct sort : List A -> List A
  ensures (xs, ys) => sorted(ys) and permutation(ys, xs)
```

The first contract permits reversal, identity, and many other functions. The second excludes identity on unsorted inputs. Neither contract, by itself, requires a particular sorting algorithm, stable ordering of elements with equal keys, or a running-time bound. Such demands need additional predicates or an explicitly modeled execution semantics.

7.2 Composition derives a real proof, not a label

Let $f : A \rightarrow B$ and $g : B \rightarrow C$. Suppose

$$\begin{aligned} h_f &: \forall x, P(x) \rightarrow Q(x, f(x)), \\ h_g &: \forall x y, Q(x, y) \rightarrow R(x, y, g(y)). \end{aligned}$$

Then their composition has the contract

$$\forall x, P(x) \rightarrow R(x, f(x), g(f(x))).$$

Its proof is the term

$$\lambda x h_P. h_g x (f(x)) (h_f x h_P).$$

The intermediate output is retained. This matters when g 's precondition refers to the result selected by f , rather than to an arbitrary object with similar printed syntax.

A common specialization has an intermediate invariant $Q(y)$ independent of x . A more general sequential contract introduces a witness together with its relation to x . Neither requires an untyped bag of guard strings. Backward planning can derive a sufficient precondition for the whole sequence, but BASALT does not promise to compute weakest preconditions for arbitrary mathematical predicates.

7.3 Variance and consequence

Suppose f has a contract with precondition P and postcondition Q . To use it at a new interface (P', Q') , it suffices to prove

$$\forall x, P'(x) \rightarrow P(x), \quad \forall x y, P'(x) \rightarrow Q(x, y) \rightarrow Q'(x, y).$$

The new interface may demand a stronger precondition, but it cannot silently claim the old implementation works for more inputs. It may advertise a weaker postcondition, but not a stronger one without a theorem. For dependent functions, the output adapter is instantiated after the input and its proof are fixed.

This directionality prevents a common cache mistake. A theorem established under $x \neq 0$ is not a stronger result than an unconditional theorem merely because it has an additional hypothesis. A higher-precision observation and a result with stronger assumptions vary in different directions.

7.4 Relational behavior and abstraction adequacy

Some properties compare two executions:

$$\text{Preserves}(f, \mathcal{R}, \mathcal{S}) := \forall x x', \mathcal{R}(x, x') \rightarrow \mathcal{S}(f(x), f(x')).$$

Examples include monotonicity, preservation of congruence modulo an ideal, nonexpansiveness, and compatibility with an observational equivalence. These contracts permit certified replacement under selected consumers without claiming the underlying objects are equal.

A useful general bridge is the following.

Proposition 7.1 (Adequacy of an observation). *Let $o : A \rightarrow O$ be an observation, $f : A \rightarrow B$ an operation, and $\hat{f} : O \rightarrow B$ an abstract implementation. If $\forall x, f(x) = \hat{f}(o(x))$, then $o(x) = o(y)$ implies $f(x) = f(y)$.*

Proof. Substitute the two adequacy equations and apply congruence of $\hat{f}$ to $o(x) = o(y)$. $\square$

For example, equality of list lengths is adequate for a function known to factor through length. It is not adequate for the identity function on lists, a sorting result, or a sum of elements. Rather than declaring an observation “safe,” a package proves which consumers respect it. This supplies a concrete interpretation of observational types without inventing a universal hierarchy of information.

7.5 Universal behavior is not finite testing or polymorphism

Testing a candidate on finitely many lists proves, at most, the tested cases. A transfer function for lengths becomes universally valid only after the semantics of the candidate’s permitted constructors and primitives have been connected to that transfer function by proof.

Nor should a Haskell-inspired synthesizer import a global parametricity principle into Lean. Lean’s official axiom discussion gives noncomputable polymorphic functions that inspect a type proposition using classical reasoning, invalidating a naive naturality theorem for every polymorphic list function [13]. A restricted syntax can support a proved parametricity or length theorem; an unrestricted Lean type does not provide that theorem automatically.

This is an especially important distinction for Leant. Structural synthesis can remain excellent at constructing candidate programs. Its abstraction’s behavioral laws need an explicit semantic interpretation before they become properties of the corresponding Lean functions.

8 Relational and scoped types: what belongs to the whole situation

8.1 Beyond a row of adjectives

A short exact sequence, a commutative square, a measure pushforward identity, and a function with a specified inverse are predicates on several subjects. They should be introduced as named relation instances:

```
let S := (A --f--> B --g--> C)
know S is short exact

let e := (eA, eB, eC)
know e is an isomorphism of diagrams from S to T
```

The first declaration does not follow from f being injective and g being surjective alone. The middle exactness condition remains necessary. The second requires commuting squares; three arbitrary objectwise isomorphisms are not enough. The compiled relation is an ordinary Lean proposition or a record whose proof fields state the required equations.

The fact index therefore uses subject tuples or a canonical diagram handle, not only one object identifier. It is a hypergraph of proved relationships in an implementation sense, but no graph algorithm is granted mathematical authority. Every edge contains, or reconstructs, a Lean proof.

8.2 Locality is an index, not a confidence score

The following are distinct predicates:

$$\begin{aligned}
 &f = g, \\
 &\forall x \in U, f(x) = g(x), \\
 &f =^{\mathcal{F}} g \quad (\text{eventual equality in a specified filter}), \\
 &f = g \quad \mu\text{-almost everywhere}, \\
 &[t^k]f = [t^k]g \quad \text{for } k < N.
 \end{aligned}$$

The last expression concerns formal series rather than arbitrary functions. These predicates do not form one linear order. In particular, almost-everywhere equality is usually not enough to preserve a value or derivative at a specified point. A finite jet supplies exact finite algebraic information, not a numerical error estimate.

Some transfers are legitimate and useful. If $V \subseteq U$, equality on U restricts to V . Equality on an open neighborhood of x yields eventual equality in $\mathcal{N}(x)$. Almost-everywhere equality under μ transports to ν when a suitable absolute-continuity theorem applies. Each transfer has its own direction and premises. A mere inclusion of printed domain labels is not evidence for the transfer.

8.3 Intersection, images, and dependent locality

Composing two on-domain equalities produces equality on the intersection of the domains. A request on a larger domain is not thereby solved. Likewise, using a property of g on V while composing $g \circ f$ on U requires $f(U) \subseteq V$. This image-containment fact is relational and depends on the selected f , U , and V .

For differentiation, a neighborhood statement about a composition may need continuity or a derivative theorem to establish the relevant image behavior. The autonomous example avoids a gratuitous global strengthening by asking its auxiliary derivative hypotheses only at values $W(x)$

for $x \in U$. The initial frontend should preserve such theorem signatures rather than replace them with an easier but stronger “every function is smooth globally” profile.

8.4 A common shape without a universal relation lattice

A package may define an evidence family

$$\text{Obs}(\vec{x}; \iota) : \text{Prop},$$

where $\vec{x}$ are its subjects and ι are indices such as a set, filter, measure, precision, branch, or selected diagram. It registers concrete weakening and composition theorems. This gives common infrastructure for search, storage, and rendering while leaving mathematical meaning in the package’s actual predicates.

A root-isolating interval and a complete algebraic solution predicate can be incomparable. Two valid interval enclosures need not be equal as intervals. Two exact presentations of the same object can be related through their meaning proofs without forcing every representation route to normalize to a single expression. Coherence is required only for the observation the caller actually uses.

9 Evidence synthesis: demand-driven, bounded, and subject-preserving

9.1 A concrete elaboration procedure

The intended procedure has two phases. First, elaborate the statement and meaning-bearing data: variable types, scalar structures, domains, relation indices, and named methods. Second, generate and solve the proof obligations required by the chosen expression or method. An author may explicitly delegate a data choice by a fixed specification; otherwise proof search must not assign meaning-bearing holes merely to make the goal easy.

For each required proposition, the initial implementation tries direct local evidence, projections from accepted refined values, and explicitly registered small implication rules. It then tries bounded theorem application, selected ordinary Lean tactics, and optional providers such as Leant or a certificate producer. The order and budgets are project policy, not mathematics.

```

elaborate application at fixed statement and revision:
  infer the selected operation and its dependent roles
  instantiate its ordinary Lean contract
  for each premise, in telescope order:
    reuse exact local evidence, if available
    otherwise request evidence under a bounded profile
    validate the returned term at this exact premise
  build the application from the accepted arguments
  check the result at the original requested type
  commit the source edit only if the revision still matches

```

This describes a procedure, not a completeness claim. Failure reports the unsolved premise or exhausted route. It does not mean that the theorem is false.

9.2 Index rules by the conclusion’s subjects and mathematical role

A rule for continuity of a CDF should be discoverable from a requirement about $\text{cdf}(\mu)$ and refer back to the same μ . A rule for composition of ideal-preserving maps should identify its source

ideal, intermediate ideal, and target ideal. A rule for exactness transport should identify both arrows and the square witnesses.

These roles are annotations on stable project-owned theorem wrappers. They are not inferred indefinitely from fragile binder positions or from English names. The underlying theorem remains callable in ordinary Lean. A wrapper signature change is checked as an interface change, not silently accepted because a similarly named declaration still exists.

9.3 Why not saturate the context with every consequence?

Unrestricted forward closure is both unbounded and undesirable. From one property, there may be infinitely many consequences and many equivalent formulations. Aggressive expansion also obscures which assumptions a fact actually uses.

The preferred implementation performs demand-driven backward search with memoization, supplemented by a very small collection of cheap forward rules. A memo entry is indexed by the elaborated proposition, its ordered context, relevant environment identity, and policy. A proof term, not a Boolean “known” flag, is the authority. Exact identity is the cheap reuse path; reuse across changed contexts needs checked instantiation and any required transport.

9.4 Guards, cycles, and ordinary induction

A conditional theorem $G \rightarrow Q$ can be recorded even when G is not known. It cannot be used to establish Q while simultaneously using that same unproved Q to establish G . A guard solver detects such unresolved cycles and reports them rather than manufacturing evidence.

This does not outlaw mathematical induction or recursive definitions. An induction hypothesis is a legitimate scoped premise supplied by an induction theorem or a well-founded recursion principle. The distinction is between a checked rule authorizing recursive reasoning and a circular chain of unresolved elaboration obligations with no such rule.

9.5 Reuse and maintenance

A derived property records the terms and hypotheses it actually depends on. Changing a measure invalidates properties about that measure; renaming its printed binder need not. Changing a selected topology may invalidate analytic properties even when the carrier has the same name. Changing a proof of an unchanged proposition need not change the mathematical subject.

For cross-revision reuse, hashes identify candidates for reuse, not semantic proofs. An adapter must check that the current target follows from retained evidence under the current hypotheses and policy. The dependency representation should begin as an index into Lean’s existing context and declaration data, not as a second, independently authoritative environment.

9.6 Diagnostics should name the mathematical obstruction

A useful failure is not “cannot synthesize instance 37.” It is, for example:

The null-boundary step requires that this measure assigns zero mass to $\{x\}$. The context contains the probability and reflection laws, but no proof of that boundary condition. The calculation is valid up to the open-half-line expression; the claimed closed-half-line equality remains open.

This message can be generated from typed roles and the unresolved proposition. It need not be a free-form explanation invented after the fact. Expanded views should expose the exact Lean target behind every such description.

10 Worked case I: probability, atomlessness, and CDF symmetry

Let μ be a probability measure on $\mathbb{R}$, with no atoms, and let $r_a(t) = a - t$. Suppose $(r_a)_*\mu = \mu$. Write $F(x) = \mu((-\infty, x])$. The ProveIt theorem `cdf_reflection_sub` establishes

$$F(a - x) = 1 - F(x) \quad \text{for every } x \in \mathbb{R}$$

under these hypotheses [3].

10.1 A complete mathematical calculation

The reflection map is measurable, and $r_a^{-1}((-\infty, a - x]) = [x, \infty)$. Therefore

$$\begin{aligned} F(a - x) &= \mu((-\infty, a - x]) \\ &= ((r_a)_*\mu)((-\infty, a - x]) \\ &= \mu([x, \infty)) \\ &= 1 - \mu((-\infty, x)) \\ &= 1 - \mu((-\infty, x]) \\ &= 1 - F(x). \end{aligned}$$

The fourth equality uses total mass one and measurability. The fifth uses $\mu(\{x\}) = 0$. These are different proof obligations, even though a human usually reads them without pausing.

BASALT would register a probability refinement on μ , an atomless refinement on μ , and a reflection-invariance relation involving r_a and μ . The calculation requests the appropriate consequences, rather than expanding every property at declaration time.

```
let mu : Measure Real satisfying probability and atomless
let F := cdf mu
assume pushforward(reflection(a), mu) = mu

calculate F(a - x)
  = mass(mu, [x, infinity))      by pushforward reflection(a)
  = 1 - mass(mu, (-infinity, x)) by probability complement
  = 1 - F(x)                    by null-boundary replacement
```

The first displayed step abbreviates the definitional bridge from CDF to half-line mass. Its retained evidence must include that bridge and the preimage identity. This is exactly where statement-oriented code and calculation-oriented presentation need a common source-to-evidence map.

10.2 A negative neighbor: reflection without atomlessness

Take $\mu = \delta_0$ and $a = 0$. This is a reflection-invariant probability measure, but $F(0) = 1$. The proposed symmetry at zero would assert $1 = 0$. The missing null-boundary proof is essential, not bureaucratic overhead.

A well-designed interface can still return the valid statement

$$F(a - x) = 1 - \mu((-\infty, x)),$$

or equivalently the correct closed-half-line formula with the atom correction

$$F(a - x) = 1 - F(x) + \mu(\{x\}).$$

For the latter, decompose $(-\infty, x]$ into its open part and singleton. The original stronger request is not discharged unless the correction term is shown to vanish. This is a useful positive alternative, rather than a system that merely refuses every difficult step.

10.3 A second theorem: derivative behavior determines the measure

The same source proves that if a function f satisfies

$$\forall x \in \mathbb{R}, \quad \text{HasDerivAt}(F, f(x), x),$$

then μ is Lebesgue measure with density f , expressed in Lean by `withDensity` using `ENNReal.ofReal(f(x))` [3]. Its proof uses monotonicity of the CDF to derive $f \geq 0$ and interval integrability of the derivative, applies the fundamental theorem of calculus on each interval, and identifies measures on half-open intervals.

An enriched declaration can express the whole derivative relation:

```
let f : Real -> Real
know derivative(F) is f at every real point

claim mu = lebesgue.withDensity(f)
  by agreement on half-open intervals using the fundamental theorem
```

The phrase “at every real point” belongs in the public contract. It cannot be dropped in favor of “almost everywhere.” The step CDF of δ_0 has derivative zero away from zero, hence almost everywhere, but δ_0 is not the zero-density measure. Continuity or differentiability properties must therefore carry their exact quantifier and locality indices.

10.4 Where the conciseness comes from

This example does not need an exotic new prover. It needs theorem packages for CDF observations, local evidence about the selected measure, and a calculation interface that keeps endpoint and quantifier distinctions visible. The gain would come from not reassembling the same contextual prerequisites at every step. Whether that gain exceeds what good Lean wrappers alone provide is an empirical question reserved for the matched evaluation.

11 Worked case II: ideal-preserving maps and completion functoriality

Fix a commutative scalar ring k . Consider commutative k -algebras R, S, T with selected ideals I, J, K . The inspected FLT source constructs the maps below explicitly, including their quotient-level compatibility and their composition laws [6].

11.1 A useful dependent morphism type

Define the interface

$$\text{IdealHom}_k((R, I), (S, J)) = \{f : R \rightarrow_{k\text{-alg}} S \mid f(I) \subseteq J\}.$$

The notation $f(I) \subseteq J$ can be implemented by the existing ideal-map inclusion predicate. The selected ideals are part of the type’s parameters. The interface does not bundle an arbitrary new ideal with each use of f .

Proposition 11.1 (Composition of ideal-preserving maps). *If $f(I) \subseteq J$ and $g(J) \subseteq K$, then $(g \circ f)(I) \subseteq K$.*

Proof. For $r \in I$, the first premise gives $f(r) \in J$ and the second gives $g(f(r)) \in K$. The composite is the ordinary composite algebra homomorphism. $\square$

Consequently composition returns a value of the enriched morphism type, with its proof field obtained by composition of the two preservation proofs. This is a small behavioral theorem, not a new algebraic operation.

An initial wrapper can expose this proof in ordinary Lean. The new syntax is justified only if it makes repeated use of the wrapper easier to author, understand, or maintain.

11.2 From ideal preservation to maps on quotient towers

For every n , the homomorphism sends I^n into J^n : products of n elements of I map to products of n elements of J , and the generated ideal is respected. Thus it induces

$$f_n : R/I^n \longrightarrow S/J^n, \quad [r] \longmapsto [f(r)].$$

The preservation condition is exactly what makes this representative formula well defined. For $m \leq n$, let q_{nm}^R and q_{nm}^S be the quotient transition maps. Then

$$q_{nm}^S \circ f_n = f_m \circ q_{nm}^R,$$

since both sides take the class of r to the class of $f(r)$. The source proves corresponding identities by passing to a representative and reducing the definitions [6].

These equations form a *relation on the whole family* $(f_n)_n$. They should be retained as a tower-morphism contract, not rediscovered from unrelated local facts at each use. A map between the carrier types of two levels, even if bijective, is not automatically a morphism of towers.

11.3 Completion as a contract-preserving construction

In the compatible-system description of the completion, an element is a family $x_n \in R/I^n$ satisfying the transition equations. Applying f_n to each coordinate preserves those equations, so it defines

$$\widehat{f} : \widehat{R}_I \longrightarrow \widehat{S}_J.$$

The source implements the construction with its existing completion lifting interface and evaluation maps, rather than with a new definition of completion [6]. A BASALT package should wrap that interface and its characterizing equations.

Proposition 11.2 (Functoriality by observations). *For ideal-preserving algebra maps f and g ,*

$$\widehat{g \circ f} = \widehat{g} \circ \widehat{f}.$$

Proof. At each quotient level n , both sides send the coordinate represented by r to the coordinate represented by $g(f(r))$. The quotient-level composition law gives equality of their evaluations for every n . Extensionality of completion elements then gives equality of the maps. $\square$

The phrase “for every n ” is essential. Checking a finite prefix of the tower supplies a finite observation, not the functor law.

```

let f : IdealHom(k, (R,I), (S,J))
let g : IdealHom(k, (S,J), (T,K))

claim complete(g after f) = complete(g) after complete(f)
proof by all quotient observations
  fix n
  both maps send [r] to [g(f(r))]
end

```

The final sentence is an abbreviation for quotient induction and the characterizing equations. Those are the method contract. An untrusted planner can fill the term details, but cannot replace the universal observation with sample computations.

11.4 Levelwise equivalence needs compatibility

The same source constructs a completion equivalence when the induced map is bijective at every level. Compatibility of the inverse maps follows from compatibility of the forward maps and injectivity at the lower level. To see this, apply f_m to the proposed equation

$$q_{nm}^R f_n^{-1}(y) = f_m^{-1} q_{nm}^S(y).$$

Naturality makes both images equal to $q_{nm}^S(y)$, and injectivity of f_m finishes the proof. The inverse coordinate family is therefore compatible.

When bijectivity is supplied only propositionally, constructing an inverse may use noncomputable choice. The source explicitly uses noncomputable definitions. A contract promising an equivalence should not thereby advertise an executable inverse algorithm. Conversely, a caller requesting an executable inverse must supply or synthesize an appropriate computational presentation.

11.5 Negative neighbors and their typing failures

For the identity ring map on $\mathbb{Z}$, choose $I = (2)$ and $J = (3)$. The condition $I \subseteq J$ fails. The representative formula from $\mathbb{Z}/(2)$ to $\mathbb{Z}/(3)$ would send the equal classes of 0 and 2 to unequal classes. The enriched morphism type correctly prevents constructing this quotient map.

A forward preservation condition for an algebra equivalence does not imply the reverse preservation condition for its inverse. Similarly, arbitrary objectwise equivalences of quotient levels need not commute with transitions. Neither omission is repaired by displaying the word “equivalence.”

The desirable gain is the automatic composition of *available* behavioral proofs. It is not automatic discovery of an entire completion theorem from a coarse function type.

12 Worked case III: exactness is a relational type

The inspected FLT proof models an induced representation using a tensor product with a free module and proves that this model is compatible with maps. It then derives preservation of short exactness [7]. This is a strong test because it combines chosen representations, mathematical properties of modules, and relations on diagrams.

12.1 The abstract ladder argument

Let

$$0 \longrightarrow M_1 \xrightarrow{f} M_2 \xrightarrow{g} M_3 \longrightarrow 0$$

be a short exact sequence of k -modules and let F be a flat k -module. Suppose another diagram S has objects S_1, S_2, S_3 and maps f', g' , and there are linear equivalences

$$e_i : S_i \xrightarrow{\sim} F \otimes_k M_i \quad (i = 1, 2, 3)$$

such that

$$\begin{aligned} e_2 \circ f' &= (1_F \otimes f) \circ e_1, \\ e_3 \circ g' &= (1_F \otimes g) \circ e_2. \end{aligned}$$

The sequence obtained by tensoring is short exact because F is flat. The equivalences and commuting squares transport its three components: left injectivity, middle exactness, and right surjectivity.

For middle exactness, take $b \in S_2$ with $g'(b) = 0$. Commutativity puts $e_2(b)$ in the kernel of $1_F \otimes g$. Tensor exactness supplies u with $(1_F \otimes f)(u) = e_2(b)$. Surjectivity of e_1 gives a with $e_1(a) = u$. The first square and injectivity of e_2 yield $f'(a) = b$. The reverse inclusion follows from the zero-composition law. Injectivity and surjectivity are transported by analogous applications of the squares and the equivalences.

This is the mathematical content of the source's ladder helper. The actual application uses the free module of finitely supported scalar functions on the group, so the flatness requirement is discharged by the relevant module structure rather than by strengthening the base ring to a field [7].

12.2 A diagram-oriented interface

```
let X be a short exact sequence of k-representations
let U := underlying_module_sequence(X)
let F := finitely_supported_functions(G, k)
know F is flat                                by its free-module structure

let T := tensor(F, U)
know T is short exact                          by flat tensor transport

let e := induced_representation_model(X)
know e is a diagram isomorphism from underlying(indBot(X)) to T
    by naturality on generators

conclude indBot(X) is short exact    by diagram transport
```

The word “naturality” generates the two required squares. A method for checking them on generators needs a spanning or induction principle; it is not permission to test a few sample vectors. The source explicitly develops such a generator-based induction and its formulas for the modeling isomorphism [7].

The relevant property types are therefore relational:

$$\text{ShortExact}(f, g), \quad \text{Commutes}(e_1, e_2, f, f'), \quad \text{DiagramIso}(S, T, e).$$

Their dependencies make the larger proof readable. A flatness fact about an unrelated module or over a different scalar ring does not satisfy the tensor step.

12.3 What a unary system would lose

Even for a short complex, injectivity of the first arrow and surjectivity of the second are insufficient. Over a nonzero field, consider

$$k \longrightarrow k^3 \longrightarrow k, \quad a \longmapsto (a, 0, 0), \quad (x, y, z) \longmapsto z.$$

The maps compose to zero and have the two endpoint properties, but the kernel at the middle is two-dimensional while the image is one-dimensional. A missing middle-exactness relation is a genuine mathematical gap.

Flatness also matters. Multiplication by 2 is injective on $\mathbb{Z}$, but after tensoring with $\mathbb{Z}/2\mathbb{Z}$, the corresponding endomorphism is zero on a nonzero module and is not injective. A generic “tensor preserves properties” label would therefore be wrong. The exact preservation theorem and its hypotheses must determine what the language infers.

12.4 An honest compression account

The final source theorem already uses its prepared ladder helper compactly. The new language should not claim to save the helper’s mathematical work by hiding it behind one keyword. Its plausible contribution is to make the helper’s reusable interface explicit, to discover the correct instances and squares, and to expose missing relational prerequisites before low-level coercion mismatches dominate the error message.

13 Control case: range-local analytic induction

The autonomous-derivative module is deliberately retained as a control from the earlier rounds. Let U be open, $W' = \phi \circ W$ on U , and suppose a family G_n has derivatives G'_n at all values $W(x)$ for $x \in U$, with

$$G_0(W(x)) = W(x), \quad G_{n+1}(W(x)) = G'_n(W(x))\phi(W(x)).$$

Then $W^{(n)}(x) = G_n(W(x))$ on U . This is the contract of the inspected source theorem, with derivative existence represented explicitly [4].

A proposed proof reads:

```
claim iterated_derivative(n, W) = G(n) after W on U
proof by induction on n, retaining the domain U
  zero: use the initial identity
  successor:
    differentiate the previous identity locally on U
    use the chain rule at values attained by W
    use the recurrence for G
end
```

The induction hypothesis has an on- U index. At $x \in U$, openness permits its conversion to eventual equality in $\mathcal{N}(x)$. The local derivative transfer theorem then applies, followed by the chain rule and the recurrence. The range-local derivative contract is sufficient; global differentiability of every G_n would be an unnecessary strengthening.

The negative neighbor is $f(t) = t$ and $g(t) = 0$ at $t = 0$. Equality of their values at zero cannot justify replacing their derivatives. A second negative neighbor removes derivative existence and keeps only a displayed equation using Lean’s total derivative operator; such an equation does not provide the `HasDerivAt` premise needed by the chain rule. The system must request the stronger evidence rather than silently reinterpret the notation.

This example tests whether the refinement layer improves use of an already well-factored theorem. It does not establish that new type syntax is necessary: an annotated Lean wrapper may perform equally well.

14 Leant integration: from structural candidates to proved behavior

14.1 What was inspected, and what the code actually promises

The inspected Leant revision is `3a40904be8a410d832d9ab6900b3d3e7b425eccb`, the same revision cited by the round-two reports. The additional finding here is not a later version; it is functionality beyond the narrow verification and tactic-suggestion discussion highlighted in the syntheses.

The verification module’s opaque `Verified` wrapper records acceptance by a supplied callback. It explicitly disclaims behavioral evidence, a solver certificate, and a kernel proof. The engine exposes exact typed origins and retained semantic sidecars. The length-contract vocabulary describes passive assertions, including explicitly supplied provider laws. The length handoff then binds one accepted candidate occurrence to a candidate-specific finite-spine problem using its retained origin and exact rendering [8, 9, 10, 11].

This is useful existing infrastructure. It should not be dismissed as merely “a string suggester,” nor promoted to an original-target behavioral theorem. The handoff’s stated guarantee is checked structural correspondence at its particular boundary. Its refusal cases include changed targets, missing typed origins, renderer changes, and unavailable or ambiguous provider identities. Those distinctions should survive the frontend integration.

14.2 The missing bridge for theorem-level behavioral claims

An asserted provider law such as “this provider preserves length” is not a proof about the corresponding Lean function. A BASALT adapter should require a theorem witness indexed by that exact function. For a primitive u , an example law is

$$h_u : \forall xs, \text{length}(u(xs)) = L_u(\text{length}(xs)).$$

The adapter then needs a theorem relating the permitted candidate syntax to its denotation and a theorem that the abstract length interpretation matches that denotation. Finally it must connect the denoted candidate to the actual Lean expression inserted at the target.

The chain is

$$\begin{aligned} \text{exact candidate origin} &\longrightarrow \text{typed candidate syntax} \\ &\longrightarrow \text{Lean denotation} \longrightarrow \text{behavioral theorem.} \end{aligned}$$

The first arrow may be an integration invariant supported by structured receipts; the latter semantic implications require actual proofs before the whole result can be advertised as a Lean theorem. Naming every stage “checked” would conceal precisely the distinction the source preserves.

14.3 A restricted length calculus with an explicit soundness argument

Consider a finite first-order grammar

$$e ::= x_i \mid [] \mid \text{reverse}(e) \mid e_1 ++ e_2, \quad 1 \leq i \leq r.$$

Let $\llbracket e \rrbracket(\vec{x})$ be its ordinary list semantics. Define an abstract interpretation $L(e) \in \mathbb{N}^r$ by

$$L(x_i) = \mathbf{e}_i, \quad L([]) = 0, \quad L(\text{reverse}(e)) = L(e), \quad L(e_1 ++ e_2) = L(e_1) + L(e_2).$$

Proposition 14.1 (Length interpretation). *For every expression in this grammar and every list input vector,*

$$\text{length}(\llbracket e \rrbracket(\vec{x}s)) = \sum_{i=1}^r L(e)_i \text{length}(xs_i).$$

Proof. Induct on e . The input case selects the corresponding length, and the empty list has length zero. Reversal preserves length. In the append case, the length of the result is the sum of the two lengths, so substitute the two induction hypotheses and distribute the finite sum. $\square$

The mathematical proof identifies exactly which primitive laws a Lean formalization needs. Extending the grammar with another primitive, recursion, or case analysis requires extending the denotation and the proof; a recognizable function name is not a substitute.

The companion implements this restricted interpreter and tests finite instances. It is not an implementation of the full Leant candidate language, and the proposition above has not been kernel-checked in this work.

14.4 A subtle negative-evidence issue: realizable abstract inputs

Even an exact length calculation does not automatically make every abstract counterexample a concrete input. For a fixed element type A , the set

$$V_A = \{\text{length}(xs) : xs : \text{List } A\}$$

is $\{0\}$ when A is empty, and all of $\mathbb{N}$ when A is inhabited. An arithmetic counterexample requiring an input of length one cannot refute a claim about lists over an empty type.

Thus a counterexample provider needs both an abstract violating assignment and a *realization* by inputs of the original types. A concrete element of A can build lists of the required lengths; a mere unsupported assumption that such an element exists cannot. Positive abstract proofs can remain sound without every abstract input being realizable, but negative evidence requires this additional bridge.

This is an important place for dependent behavioral types to help. The counterexample result should contain a concrete input and a proof that it violates the candidate’s contract, rather than a detached vector of lengths. A refutation of one candidate is also not a refutation of the existence of some other candidate satisfying the synthesis request.

14.5 Correlated outputs and relation strength

The inspected source also contains a binary-product finite-spine handoff [11, 10]. For a pair of output lists, a behavioral contract should preserve correlations involving the same input, rather than store two unrelated “length-like” labels. For example,

$$\text{length}(ys) + \text{length}(zs) = \text{length}(xs)$$

is different from two independent upper bounds. The interpretation theorem must match the exact product representation and source-ordered roles.

Length alone does not prove permutation, content preservation, sorting, or identity. Both reversal and identity preserve length, and identity fails a sorting request on $[2, 1]$. These are not defects in a length domain; they are invalid promotions between observations. The frontend should make the accepted behavioral predicate explicit.

14.6 One provider envelope, several mathematical authorities

A shared provider envelope can carry the request identity, ordered context, source revision, selected policy, proposed term or certificate, and diagnostic status. It need not impose one universal certificate language. A proof term, a polynomial certificate, a typed length derivation, and a counterexample have different semantic checking theorems.

For tactic edits, the actual displayed edit must be replayed in its original surrounding context after insertion. For data synthesis, the selected value and its specification must be checked. For negative outcomes, distinguish unsupported input, bounded failure, nonderivability in a stated calculus, refuted candidate, and a checked negation of the original target. This is a proposed adapter contract; this article does not claim to have re-audited every current tactic-suggestion path in `Main.hs`.

15 Certified CAS: a concrete multivariate checker, not a slogan

A hybrid system should use computation to discharge mathematical obligations whose guarantees fit the current refined type. The first verified core need not be a complete CAS. A small polynomial certificate checker is enough to exercise coefficient domains, reification, execution, and exact-target reconstruction.

15.1 Finite representation and denotation

Fix a number of variables d . A monomial exponent is a vector $\alpha \in \mathbb{N}^d$. Represent an integer polynomial by a finite coefficient map $c : \mathbb{N}^d \rightarrow \mathbb{Z}$, stored as a sorted list of nonzero coefficients with unique exponent vectors. The zero polynomial has an empty support. A parser verifies that every exponent vector has length d , every exponent is a natural number, and every coefficient is an integer.

For a commutative ring R and a valuation $v : \{1, \dots, d\} \rightarrow R$, define

$$\llbracket c \rrbracket_v = \sum_{\alpha \in \text{supp}(c)} \iota_R(c_\alpha) \prod_{j=1}^d v_j^{\alpha_j},$$

where $\iota_R : \mathbb{Z} \rightarrow R$ is the integer scalar map. No injectivity of ι_R is assumed. Empty products equal one, so the definition also handles $d = 0$.

Addition merges equal exponent vectors and adds their coefficients. Multiplication forms all pairs of terms, adds exponent vectors, multiplies coefficients, and normalizes the resulting list. Normalization groups duplicate exponent vectors and deletes zero coefficients. These are concrete finite algorithms, not operations on an unspecified semantic polynomial type.

15.2 Why the arithmetic operations preserve denotation

Lemma 15.1 (Normalization). *Grouping terms with the same exponent vector, replacing their coefficients by the sum, and removing zero coefficients does not change denotation.*

Proof. For one exponent vector α , distributivity and additivity of ι_R give

$$\sum_i \iota_R(a_i) \prod_j v_j^{\alpha_j} = \iota_R\left(\sum_i a_i\right) \prod_j v_j^{\alpha_j}.$$

Regrouping a finite sum in a commutative additive group preserves its value, and zero terms contribute zero. Apply this independently to each group. $\square$

Lemma 15.2 (Addition and convolution). *For the finite operations just described,*

$$\llbracket c + c' \rrbracket_v = \llbracket c \rrbracket_v + \llbracket c' \rrbracket_v, \quad \llbracket c * c' \rrbracket_v = \llbracket c \rrbracket_v \llbracket c' \rrbracket_v.$$

Proof. Addition follows from the normalization lemma. For multiplication, expand the product of the two finite sums. Multiplicativity of ι_R and the power law in a commutative ring identify a pair of monomial terms with the monomial whose exponent vector is their sum. Normalize the resulting finite list using the preceding lemma. $\square$

These proofs require commutative multiplication. A noncommutative expression must use a different representation and theorem, not a permissive coercion to this checker.

15.3 The certificate and its soundness theorem

Given polynomials $p, f_1, \dots, f_m$, a certificate is a vector of polynomials $q_1, \dots, q_m$. The checker verifies the exact coefficient identity

$$p = \sum_{i=1}^m q_i f_i.$$

The arity m must agree. A parser must not silently truncate mismatched input lists with a zip operation. A typed implementation can use vectors indexed by the same finite type after validating the external representation.

Theorem 15.3 (Denotational soundness of the specified checker). *If the described checker accepts $(p, \vec{f}, \vec{q})$, then for every commutative ring R and valuation v ,*

$$\llbracket p \rrbracket_v = \sum_{i=1}^m \llbracket q_i \rrbracket_v \llbracket f_i \rrbracket_v.$$

In particular, if $\llbracket f_i \rrbracket_v = 0$ for every i , then $\llbracket p \rrbracket_v = 0$.

Proof. Acceptance means that the two canonical coefficient maps are identical. Identical maps have identical denotations by the definition of the finite sum. Repeatedly apply the addition and convolution lemmas to the right-hand map. Under the vanishing hypotheses, each summand is zero. $\square$

This is a universal mathematical soundness proof of the specified algorithm. It is not a claim that the Python implementation is verified or that this proof has been accepted by Lean. A Lean implementation must formalize the finite representation and these lemmas, connect its Boolean equality test to coefficient-map equality, and check the actual computation.

15.4 The original-target bridge

Suppose the caller's goal concerns an expression $e : R$ and hypotheses $h_i : e_i = 0$. Reification must produce p, f_i and proofs

$$\rho : e = \llbracket p \rrbracket_v, \quad \rho_i : e_i = \llbracket f_i \rrbracket_v.$$

Given a checked acceptance proof and the soundness theorem, rewrite the hypotheses through ρ_i , obtain $\llbracket p \rrbracket_v = 0$, and transport through ρ to conclude $e = 0$.

Opaque subexpressions may become variables in v , but their identities must be exact elaborated terms in the original context. Two atoms with the same printed name in sibling scopes are not interchangeable. If two syntactically different atoms are identified by a theorem, the

reification bridge must include that theorem. Coefficient-domain casts require the appropriate homomorphism laws, not a heuristic string conversion.

A suitable Lean contract has the following shape; the names are illustrative:

```
check_sound :
  check input certificate = true ->
  forall (R : Type) [CommRing R] (v : Fin d -> R),
    denote target v = sum_i (denote multiplier[i] v *
                             denote generator[i] v)
```

The consumer supplies the reification equalities and actual vanishing hypotheses. The checker does not need to understand the caller's whole theorem.

15.5 Execution is another proof obligation

A correctness theorem about a Lean function does not certify bytes returned by an unchecked executable. The actual assertion $\text{check}(i, c) = \text{true}$ must be obtained through the chosen trusted execution route: kernel reduction, checked reconstruction, or an explicitly approved computation mechanism. Lean's validation documentation separates statement interpretation, axioms, and proof checking; publication must account for each [14].

Native evaluation is not authorized merely because it is faster. A project's allowed axiom inventory and execution assumptions are a prior policy choice; performance selects among the routes that policy permits. Toolchain upgrades require a new audit of the transitive dependencies, rather than renaming an old allowlist entry. The source reports' native-versus-kernel timings should not be generalized to a different checker or machine [1, 2].

15.6 Limits that the type of the result should expose

The certificate proves an ideal-membership consequence. Failure to find a certificate is not nonmembership. A certificate for p^n does not in a general ring prove $p = 0$: $2^2 = 0$ in $\mathbb{Z}/4\mathbb{Z}$, but $2 \neq 0$. Such a promotion needs an appropriate reducedness or power-reflection theorem.

A coefficient extension may preserve an identity without reflecting a richer existence claim. For example, X belongs to the ideal generated by $2X$ in $\mathbb{Q}[X]$, using multiplier $1/2$, but not in $\mathbb{Z}[X]$. A rational certificate cannot be read as an integer-coefficient witness. Likewise, equality of polynomial functions over a finite field need not be equality of polynomials: $X^2 - X$ is nonzero as a polynomial over $\mathbf{F}_2$ but vanishes at every field element. The requested relation determines which checker is appropriate [1, 2].

15.7 What the companion actually demonstrates

The Python companion implements the sparse integer representation, its normalization and convolution, exact-arity certificate checking, and the restricted length interpreter of Section 14. Its 24 test methods passed in this environment. They include valid multivariate certificates, corrupted coefficients, mismatched arities, malformed exponents, a finite-jet residual, and concrete counterexamples for invalid property promotions.

These tests are finite implementation demonstrations. They do not formalize reification of Lean expressions, validate Leant's actual candidate semantics, measure Lean checker performance, or establish usability. The universal arguments in this section are written mathematical proofs; the missing kernel formalization remains an explicit first implementation deliverable.

16 Observation-indexed contracts connect jets and quotient towers

The formal-series proposals in round two already developed demand-driven precision planning. The completion example suggests a wider application of the same idea: make precision behavior a relational contract on operations, not a special heuristic inside a CAS [1, 2].

16.1 A general sufficient-demand interface

Suppose each object type A has registered observation relations $E_A(n, x, y)$, meaning that x and y agree at the observation level n . For an operation $f : A \rightarrow B$, a sufficient-demand transformer is a function $d_f : \mathbb{N} \rightarrow \mathbb{N}$ with a proof

$$\forall n \, x \, y, \quad E_A(d_f(n), x, y) \rightarrow E_B(n, f(x), f(y)).$$

The interface says how much agreement at the input suffices for a requested agreement at the output. It need not be optimal. Monotonicity or restriction rules for observations are additional theorems when needed by a planner.

Proposition 16.1 (Composition of precision contracts). *If $f : A \rightarrow B$ has sufficient-demand transformer d_f and $g : B \rightarrow C$ has d_g , then $g \circ f$ has transformer*

$$d_{g \circ f}(n) = d_f(d_g(n)).$$

Proof. Input agreement at $d_f(d_g(n))$ gives agreement of the two f -outputs at $d_g(n)$ by the contract for f . The contract for g then gives output agreement at n . $\square$

The order of composition is important: the consumer's demand propagates backward through g before f . For multi-input operations the transformer returns a vector of sufficient demands, with any guards and shared parameters retained in their actual scope.

16.2 Three concrete instances

For formal series, let $E(n, A, B)$ mean agreement of coefficients below degree n . The formal derivative has sufficient demand $d_D(n) = n+1$, since the coefficient of degree k after differentiation uses the input coefficient of degree $k+1$. This is a formal algebraic statement, not a theorem about the analytic derivative of an arbitrary function.

For the ideal-preserving maps in Section 11, let $E_R(n, x, y)$ mean $x - y \in I^n$ and similarly for S, J . The induced map has demand $d_f(n) = n$, by preservation of powers of ideals. Its extension to completion observations satisfies the corresponding quotient-level contract.

For truncated polynomial multiplication, agreement below degree n in each input suffices for agreement below degree n in the product. More precise rules can use known valuations, but they require their own proved hypotheses. The planner must not infer a valuation merely from a coefficient list that was never connected to the original series.

16.3 Residual certificates remain observations of the original root

A particularly useful inherited bridge starts with an existing formal root Δ of a polynomial equation $F(Y) = 0$ over $R[[t]]$. If J has the same constant term, the reduced derivative at that term is a unit, and $F(J)$ vanishes below degree $N \geq 1$, then J and Δ agree below degree N . Indeed,

$$F(J) - F(\Delta) = (J - \Delta)U,$$

where the constant term of U is the specified unit. Thus U is invertible, and membership of the residual in (t^N) implies membership of $J - \Delta$ in (t^N) [1, 2].

A computed J is an observation of the original Δ , not a replacement for a universally quantified root. The result's type records its precision and branch data. A demand for the first N coefficients of $D\Delta$ can then request $N + 1$ coefficients of Δ , using the derivative contract. The finite producer still needs a proof that its truncation algorithm realizes the advertised observation.

This does not collapse formal series, completion elements, analytic germs, and numerical approximations into one type. It provides a shared protocol for relation-indexed information flow. Every domain supplies its own interpretation, restriction theorems, and operation contracts.

17 Definitions, choices, and the boundary between mathematics and execution

17.1 Definition packages should expose reasons for existence

A mathematical definition package can contain a semantic object, an existence or uniqueness theorem, characterization lemmas, observations, and certified computational presentations. The package need not expose every implementation detail in the default mathematical view, but it must not introduce a hidden axiom when a construction is missing.

There are three different author actions:

```
fix Delta satisfying its defining equation
construct Delta satisfying its defining equation
choose Delta from a proved existence theorem
```

The first introduces a quantified object or an explicit hypothesis. The second generates an existence-and-construction obligation. The third uses a particular existence theorem, possibly with noncomputable choice. The compiler must retain these distinctions. A canonical constructed solution cannot simply replace the arbitrary object introduced by the first form without a uniqueness or observation theorem connecting them.

17.2 Choice by specification need not require constant prompting

Automation may choose a witness when the author explicitly asks for any witness satisfying a fixed specification. It need not ask permission for every candidate formula. What it may not do is change that specification, narrow the domain, choose a different branch, or drop a required uniqueness or completeness clause.

For example, constructing a nonnegative real square root needs both the square equation and nonnegativity. The negative root has the same square but fails the specified branch. Constructing all roots of a polynomial requires a complete solution predicate or another representation with soundness and coverage; one certified root is insufficient.

17.3 Preserve total operations when lifting Lean

Existing Lean statements use their existing total operations. A frontend must not quietly replace total real division by partial-expression semantics. For example, in the total interpretation,

$$\frac{x^2 - 1}{x - 1} = x + 1$$

fails at $x = 1$, where the left side is zero and the right side is two. A guarded equality for $x \neq 1$, a correct piecewise identity, and an equality of partial expressions on a domain are three distinct possible requests.

An opt-in partial-expression package may use a domain predicate and a value on that domain. Its simplification contracts must preserve or explicitly change the domain as well as the values. The source view should visibly select this package; it should not reinterpret an imported Lean expression merely because the partial reading is more familiar to a mathematician.

17.4 Behavioral constraints beyond output equations

A proof about a pure total function does not specify the performance of every implementation with that extensional behavior. A complexity claim needs a chosen cost semantics, for example an instrumented evaluation returning (y, c) together with a proof of $c \leq B(x)$. A memory or state invariant likewise needs a state-transition model and pre/postconditions for that model. The refinement machinery can express these contracts, but cannot infer their semantics from an ordinary function type.

External search may time out or fail even when the mathematical function being specified is total. A timeout is an operational outcome, not a partial proof or a mathematical counterexample. Noncomputable mathematical choice is also different from a failed execution: it can define an object with a checked specification without providing an executable algorithm for its values.

Design decision. Use behavioral types for mathematical input–output and relational specifications first. Add cost or state effects only together with an explicit execution semantics. Do not let a generic “verified” annotation blur these different claims.

18 Three correctness boundaries, not one green checkmark

18.1 Theorem truth

The first boundary is the Lean proposition actually proved, in its actual environment and with its actual axiom dependencies. Refinements, guards, behavioral laws, and transport proofs all contribute to this boundary. The small-core translation argument in Section 6 explains why these features can remain conservative.

Inconsistent explicit hypotheses still permit arbitrary conclusions, as in ordinary logic. The frontend does not solve mathematical consistency for every context. It must instead show which hypotheses the theorem assumes and never silently promote a desired conclusion into an assumption.

18.2 Statement and source fidelity

The second boundary is whether the elaborated proposition matches the source that the author meant to submit. A perfectly checked proof of a changed statement is not a solution to the original task. Meaning-bearing metavariables, structure selection, overloaded operations, quantifier dependencies, and branch conventions therefore need explicit treatment before proof search.

A statement seal is a readable expansion of the actual interpretation, not a certificate of human intention. The default view must show restrictions that change the claim. The fully expanded Lean statement remains available. The FLT repository’s own distinction between declaration

names and authoritative statements is a useful reminder that presentation fidelity cannot be inferred from kernel validity alone [5].

18.3 Method and workflow fidelity

The third boundary is whether the accepted artifact performs the method or edit it claims to perform. A proof may establish the right theorem without using the named argument. A tactic probe may succeed while the displayed text fails when inserted into the document. A result may be valid but stale after the user changes a witness or scope.

A named method therefore generates a structured derivation record containing its subclaims and the theorem instances linking them. Publication checks those subclaims, including unused displayed assertions. Because Lean treats proofs of a proposition irrelevantly, the proposition alone does not encode a unique explanation. Method fidelity is a validation policy on the elaborated derivation and source map, not a new axiom that proofs reveal their historical origin.

18.4 An honest status representation

A result should distinguish at least mathematical guarantee, proof-validation state, unresolved guards, source binding, and policy compatibility. For example, a theorem $G \rightarrow Q$ can be fully checked while G remains unproved at the caller. A finite observation can be fully checked while not answering an exact equality request. A proof can be valid at an old source revision while no longer eligible to modify the current file.

These distinctions need not appear as a cumbersome vector in ordinary reading. They should determine the wording and available actions. “Checked conditional result; boundary condition unresolved” is better than a green badge followed by a footnote that contradicts it.

19 Architecture and implementation plan

19.1 Lean owns the authoritative objects

The implementation should retain Lean expressions, local contexts, declaration identities, and source locations as authoritative inputs. An additional index maps subject tuples and proposition heads to existing proof handles. It also records method roles, selected interpretation packages, and retained computation certificates. It should not duplicate Lean’s entire context management.

The main components are a frontend parser, an elaboration extension, a proof-bearing property index, a registry of ordinary theorem contracts, provider adapters, and an evidence-aware renderer. The finite checker lives in the Lean library once verified. An external CAS, large-language-model planner, or Haskell synthesizer is untrusted with respect to theorem truth.

Component	Existing foundation	New obligation
Local objects and facts	Lean contexts and proof terms	Subject/role index must not retarget evidence
Mathematical structures	Existing bundled structures and instances	Make selected structures visible; reject ambiguous data choices
Property inference	Theorem application and tactics	Bounded rule policies and exact-subject reconstruction
Diagram transport	Existing exactness and naturality theorems	Checked role mappings and coherent diagram packaging
CAS acceptance	Proved checker plus execution proof	Exact reification and original-target reconstruction
Leant adapter	Typed origins and callback receipts	Proved semantic laws and behavioral interpretation
Document rendering	Source spans and elaborated information	Display the actual scope and strength of every assertion
Publication and migration	Ordinary Lean checking and dependency audit	Search-free retained evidence and new receipts after upgrades

19.2 A minimal complete slice

The first implementation should not attempt all the syntax in this article. It should support local property enrichment, one refined morphism wrapper, one relation on a diagram, and one certified computation request. Each must also be usable directly from ordinary Lean.

A useful first milestone is the following combination. Formalize the sparse polynomial checker and its denotational soundness, provide exact reification for a restricted ring-expression grammar, and use it in guarded cancellation. In parallel, package ideal-preserving algebra maps and prove their composition wrapper using the existing completion APIs. These two tasks distinguish computational certificates from theorem-level property propagation while sharing the same typed request boundary.

The first frontend commands can simply be `know`, a refined function application, and a named calculation step. A narrow default display can show the required premises and their sources. A complete prose grammar, an unrestricted planner, and a global representation search engine are not prerequisites.

19.3 Subsequent exit gates, not speculative schedules

The next gate is a CDF calculation whose probability and boundary evidence is derived from the correct measure, with the Dirac negative case diagnosed at the missing boundary condition. A third gate is the exactness ladder, with commuting squares and flatness kept explicit. A fourth connects the Leant length domain to proved primitive laws and the exact accepted Lean candidate.

Each gate requires a positive case, a nearby negative case, retained evidence, and successful checking with discovery services disabled. A source edit or library upgrade must be replayed before the gate is considered passed again. The repository snapshots use different toolchains; an implementation must pin one environment and check any ports explicitly rather than assume their compiled artifacts are interchangeable.

19.4 Safe transactional elaboration

Candidate attempts run against an immutable request snapshot. Shared metavariables are committed only after the candidate and all dependent obligations have been validated. A failed attempt restores the prior state. A late result is rejected when the originating statement or source revision has changed, even if its proof was valid in the old context.

Operational commit tokens are not mathematical propositions to be freely assumed. They are application-level permissions tied to a revision. There is no need to impose linear logic on ordinary mathematical facts merely to prevent stale editor writes. The security and workflow layer should remain separate from the reusable mathematics library.

19.5 Publication artifacts and collaboration

Publication retains source, its elaborated statements, theorem terms or checkable certificates, the relevant environment pin, and a map from displayed steps to evidence. Rediscovery scripts may be useful for editing but should not be the only retained authority. External services disappearing must not invalidate already retained mathematics.

When two branches of a document are merged, merge data choices and proof evidence separately. Two new proofs of an unchanged property usually coexist. Two changed witnesses or two conflicting representation routes require a new semantic check. A union of fact caches is not a sound merge procedure when those facts belong to different source contexts.

20 Relation to prior type-system and proof-language work

The proposal should not present refinements themselves as new. Lovas and Pfenning develop refinement sorts for logical frameworks and interpret them through proof irrelevance, including a bidirectional discipline. Ghalayini and Krishnaswami develop explicit refinement types with proof terms rather than relying only on a fixed SMT fragment, and report a Lean formalization of their system [15, 16]. These works support the choice to make evidence explicit while keeping user-facing annotations compact. They are precedents, not proofs of the correctness of BASALT.

Lean’s existing subtype mechanism supplies an immediate representation for value refinements, while ordinary dependent records and functions supply the behavioral and relational encodings. The novelty claim here is therefore limited: a proposed integration discipline specialized to mathematical workflows, developed against the cited source examples and the present Leant handoff. No claim is made that the underlying refinement calculus, Hoare-style consequence rule, certificate checking, or functoriality arguments are newly discovered mathematics.

Relative to the nine round-two proposals, the emphasis changes from the type of a *returned computation* to the type-directed accumulation and use of knowledge *throughout* an argument. The most important additions are relational diagram contracts, contextual enrichment without object replacement, proved behavioral-abstraction bridges, countermodel realizability, and the application of observation-demand contracts to both symbolic and algebraic examples. These are refinements of an already strong foundation, not a replacement for its trust boundaries [1, 2].

21 Evaluation: make the new layer earn its place

21.1 Separate library, service, and language effects

The primary study should compare four matched conditions: existing Lean; Lean with the new theorem packages; Lean with the same packages and providers; and BASALT with exactly those packages and providers. An evidence-aware rendered document view can be tested separately. Otherwise a language could claim credit for a theorem that was simply absent from the baseline.

All wrapper authoring, checker proofs, role annotations, test maintenance, and training effort count toward the cost. Record unsuccessful attempts as well as completed proofs. Match solver budgets and library access. A theorem used while designing a package is development material, even if its filename had not appeared in an earlier report.

21.2 A concrete workload portfolio

The development portfolio comprises the CDF reflection and derivative-density tasks, ideal-preserving completion maps, the short-exactness ladder, the range-local derivative control, guarded polynomial reasoning, and restricted list behavior. It spans measure-indexed properties, unary map constraints, multi-arrow relations, filter-local evidence, and semantic abstraction.

Independent maintainers should select genuinely held-out tasks only after the interfaces are frozen, excluding aliases, direct restatements, and near-duplicate downstream consequences. Some held-out tasks should deliberately remain short in ordinary Lean. The frontend should have a low-cost escape hatch rather than require unnecessary narrative scaffolding.

21.3 What to measure

The main outcome is time and effort to a correctly stated, checked, maintainable theorem, not visible line count alone. Secondary outcomes include author interventions, wrapper reuse, unresolved-obligation quality, proof-checking cost, certificate size, and maintenance effort after controlled changes. Distinguish discovery time, elaboration time, and kernel checking time.

A semantic-reading test asks readers to reconstruct the exact subjects, assumptions, quantifier scope, domain, branch, precision, and completeness of a compact statement before inspecting its expansion. A method-reading test asks which inference justified a displayed transition and which facts it required. A concise interface that systematically induces wrong answers to those questions has failed, even if it saves keystrokes.

For Leant, classify obligations into direct lookup, structural composition, behavioral checking, and original-target reconstruction. Measure it on actual obligations generated by the packages rather than assuming its general synthesis ability automatically helps every CAS request.

21.4 Paired semantic acceptance tests

The following is an implementation backlog. Only the explicitly identified finite instances in the companion were executed; this table is not a claim that a language implementation passed the suite.

Boundary	Positive case	Nearby negative or required alternative
Boundary mass	Atomless reflection gives CDF symmetry	Dirac reflection needs the atom correction; unconditional symmetry fails

Boundary	Positive case	Nearby negative or required alternative
Derivative scope	Everywhere CDF derivative supplies the source theorem's premise	Almost-everywhere derivative alone does not identify a density
Subject identity	Atomlessness evidence for the selected measure	Evidence for another measure is not reusable by matching its printed name
Quotient map	Ideal-preserving homomorphism induces every level map	Identity from the (2) quotient to the (3) quotient is not well defined
Map composition	Composed ideal-preservation proof	Missing intermediate containment leaves a guard open
Diagram transport	Objectwise equivalences and commuting squares	Equivalences alone do not establish a diagram isomorphism
Tensor exactness	Flat-module hypothesis and exact source sequence	Tensoring multiplication by 2 with $\mathbb{Z}/2\mathbb{Z}$ loses injectivity
Middle exactness	Kernel equals image	Injection, surjection, and zero composition can still leave a middle gap
Local differentiation	Equality on an open neighborhood	Equality only at the point does not transfer derivatives
Precision	Input order $N + 1$ gives derivative order N	Keeping the original precision after differentiation is unjustified
Root observation	Matching constant term and unit derivative	Wrong branch or merely nonzero nonunit derivative does not suffice
Behavior	Proved primitive laws and supported syntax	Passive provider-law metadata is not a Lean theorem
Abstraction strength	Length observer used by a length-respecting consumer	Length preservation is not sorting or extensional identity
Negative evidence	Concrete input violates the selected candidate's contract	Unrealizable length assignment over an empty element type is insufficient
Certificate shape	Exact coefficient and arity checks	Changed coefficient, wrong dimension, or a truncated multiplier list is rejected
Ring capability	Commutative-ring identity under the actual structure	Noncommutative input or rational witness for an integer ideal cannot be silently accepted
Complete solving	Soundness and coverage for the candidate predicate	A witness and coverage alone permit extraneous solutions
Enclosure	Collapsed bounds justify equality	A generic nondegenerate interval is not an exact value
Publication	All displayed claims have checked evidence	An unused false intermediate assertion still invalidates the document
Revision	Retained evidence rechecks at the current context	Stale results and sibling-scope proofs cannot commit edits

A system that rejects both neighbors does not pass. Tests must specify the intended rejection boundary and the expected unresolved proposition, not only an exit code. Some weaker outputs should be accepted under a weaker request: the correct atom-corrected CDF identity is a useful result, not a failure.

21.5 Stopping and simplification rules

The evaluation should permit a library-only outcome. If packages and providers account for the improvement, retain them and drop the additional type syntax. If property closure causes excessive search, restrict the rule set rather than hide delays behind stronger automation. If

readers confuse a refined interface with stronger mathematics than it proves, revise the display before increasing coverage.

Numeric success thresholds should be selected after a pilot estimates task variation and before the main study. There is no measured productivity figure in the present work from which to infer a credible percentage improvement.

22 Risks, unresolved questions, and the recommended next artifact

The main engineering risk is that a useful property index becomes an expensive second elaborator with unpredictable search. The main mathematical-interface risk is that a concise refinement hides a changed structure, scope, or relation. The main evaluation risk is counting the benefits of new library lemmas as benefits of syntax. The main maintenance risk is a proliferation of redundant bundled types and transport lemmas.

Mitigations are deliberately conservative: local enrichment before packaging; small, demand-driven rule sets; ordinary Lean contracts; selected data structures made explicit; relation-specific adapters; and equally equipped baselines. There is no automatic principal refinement for an arbitrary mathematical term, and no reason to expect complete inference of all useful behavioral properties.

The next artifact should be a small Lean library and elaboration prototype, not a broad grammar. It should prove one multivariate checker's soundness, connect one reified expression to its original target, and expose one ideal-preserving map composition through both ordinary Lean and refined syntax. A paired CDF boundary example and a diagram-transport example then test whether the same property mechanism generalizes beyond arithmetic.

A useful research question remains: how much relational structure should be inferred from an operation's type, and how much should be supplied by named packages? The answer should be based on failed and successful elaboration traces. A second question is how to render a contract whose actual hypotheses are weaker and more local than the user's familiar informal description. The system should preserve the weaker theorem without making its statement unreadable. Neither question requires changing the kernel first.

23 Conclusion

BASALT treats mathematical knowledge as typed, scoped evidence about stable objects. Unary refinements say which properties an object has; behavioral contracts say what a selected function does; relational contracts say how an entire diagram or collection of subjects fits together. Their composition is ordinary mathematics, expressed by ordinary Lean theorems and used by a bounded elaborator.

The motivating examples show why all three forms are needed. Atomlessness licenses a CDF boundary conversion. Ideal preservation licenses quotient and completion maps. Flatness and commuting squares license exactness transport. A length contract licenses a particular observation of a synthesized list program, but not arbitrary claims about its content, and an abstract countermodel needs realizable inputs before it becomes a refutation.

This is an extension of the language in which authors state and use types, not a proposal to weaken verification or make theorem-level equality part of kernel conversion. The CAS and synthesis services propose evidence; they do not choose the theorem being proved. The new surface is worth keeping only if it improves authoring and comprehension beyond what the same theorem packages and services already achieve in Lean.

A Answers to the third-round questions in the two syntheses

The two reports overlap substantially in their questions. The following crosswalk records concrete decisions, not a claim that every implementation issue is settled [1, 2].

Question family	Decision in this proposal	Required evidence or unresolved measurement
First complete slice	Polynomial checker plus exact reification; ideal-preserving map wrapper	Lean soundness proof, checked execution, positive/negative cases, ordinary Lean access
Checker denotation	Direct evaluation into any commutative ring	Prove normalization, convolution, Boolean equality reflection, and original-target reconstruction; a bridge to semantic polynomial types is optional extra work
Existing tactics	Use existing proof-producing arithmetic tactics as controls	Pin and measure their actual limits; bounded failure is not nonmembership or refutation
Guarantee vocabulary	Ordinary propositions with typed domain-specific indices	Start with equality, guarded equality, observations, behavioral predicates, and diagram relations
Default semantics	Preserve imported Lean total operations	Partial-expression semantics is explicitly selected and carries its domain
Author delegation	Permit witness choice under a fixed requested contract	Changed domain, branch, structure, or completeness is a semantic edit
Visible obligations	Show restrictions that change the mathematical claim	Blind reading test determines whether default rendering is sufficient
Workspace state	Index Lean’s actual objects and proofs, do not duplicate them	Measure new metadata cost and invalidation behavior
Representation routes	Start with explicitly selected theorem-backed routes	Entailment-based reuse needs checked adapters, not a universal strength score
First definition packages	Ideal-preserving maps and quotient observations; CDF properties as a second domain	Compare package-only and syntax-assisted authoring before adding broad schemas
Execution policy	Trust permission is decided before performance selection	Audit actual transitive dependencies at each version; no assumed universal speed threshold
Leant contribution	Structural composition plus restricted behavioral checking	Prove exact provider laws, candidate interpretation, and counterexample realizability; measure generated obligations
Mutation catalogue	Preserve paired tests and their distinct subcases	The present finite companion is not the implemented full-language suite
Publication and upgrades	Retain terms/certificates and statements at a pin	Recheck at the destination toolchain; issue a new receipt after semantic or policy changes
Collaboration	Merge evidence separately from changed mathematical choices	Validate contexts and selected routes after a merge; never simply union stale caches
Stopping rule	Library, adapter, or renderer alone is an acceptable result	New syntax must add measurable benefit with shared infrastructure and acceptable semantic-reading accuracy

The reports’ invitation to execute a real Lean slice is appropriate. This article does not claim to have met that gate: no Lean compiler was available in the working environment. It instead supplies a more explicit mathematical checker specification, a small uncompiled proof-term core,

and executable finite demonstrations with their scope stated. A kernel-checked prototype is the next concrete deliverable, not an accomplishment inferred from this report.

B A minimal ordinary Lean core, with its validation status

The companion `ContractCore.lean` illustrates the logical encoding without new syntax, external solvers, or a dependency on Mathlib. It contains no `sorry` or added axiom. It was *not compiled here*; absence of those strings is not evidence that Lean accepted it. The following excerpt is original illustrative code, not copied from a repository.

```
structure LawfulFn (A : Type u) (B : Type v)
  (pre : A -> Prop) (post : A -> B -> Prop) where
  run : A -> B
  law : forall x, pre x -> post x (run x)

def composeLaw
  (f : A -> B) (g : B -> C)
  (P : A -> Prop) (Q : A -> B -> Prop)
  (R : A -> B -> C -> Prop)
  (hf : forall x, P x -> Q x (f x))
  (hg : forall x y, Q x y -> R x y (g y)) :
  forall x, P x -> R x (f x) (g (f x)) :=
  fun x hx => hg x (f x) (hf x hx)
```

This core also includes value-preserving refinement weakening, intersection packaging, the contract consequence rule, and observation-based congruence. These definitions illustrate why the type-system proposal can remain conservative. They are not an implementation of the fact index, parser, provider protocol, or polynomial checker in Lean.

C Source register and scope of inspection

All sources were accessed on 6 September 2026. Repository heads were resolved through the GitHub connector; the following commits identify the observed snapshots. Individual blob identities provide finer provenance for the files actually used. No repository was modified.

Repository	Observed commit
Brainstorm	b052ec011beb96a7c48df2a9988823f0f779b445
ProveIt	24ce8bd743eaab64a91ce90725ea00f498d319d2
Leant	3a40904be8a410d832d9ab6900b3d3e7b425eccb
FLT	aa2d8b34692b16c70f699536de0d8e75b9a3e9ef

C.1 Both requested syntheses

`docs/round-2/synthesis/unified-report.tex`, blob `ae7c21dac1ba615e36dff07ea6f6a1c433a40108`, was read through its substantive sections, corrections, questions, appendices, and bibliography. `docs/round-2/unified_report/unified_report.tex`, blob `de10daae20af19285ee07a476caa94e577f7d7d9`, was likewise read closely. The original nine proposals are discussed through these syntheses; this work does not claim an independent full reading of all nine reports or reproduction of their source inventories and experiments.

C.2 Provelt source modules

`Analysis/FabiusFunction/Lean/FabiusFunction/ContinuousCDF.lean`, blob 2306bd53bd46d145b4e156c065b058009ce16fdc, was read in full. The relevant declarations are `continuous_cdf_of_nullSingleton`, `cdf_reflection_sub`, and `measure_eq_withDensity_of_cdf_hasDerivAt`.

`Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean`, blob 1422b87f31073fed898ff0c8a080ea0a798aef5e, was read in full. The main interface is `AutonomousODE.iteratedDeriv_eq_comp`, with its more globally differentiable corollary used only as a comparison.

C.3 FLT source modules

`Definitions/Def_AdicCompletionRingFunctoriality.lean`, blob 13acc67dd2b0114d0f1189584b847748e6e59654, was read in full. The relevant interfaces include `levelMap` and completion-map constructions (the source uses subscripted algebra-map names), their evaluation and composition lemmas, `map_comp_le`, and the equivalence constructions from compatible levelwise bijections.

`P2M/Sol/S_Rep_indBotSC_shortExact.lean`, blob 63caa90be75bb6875f1211a8e81230f00931fe5f, was read in full. The relevant parts are the generator-based modeling equivalence, its `e_square` naturality theorem, `shortExact_of_ladder`, and the final `solution` theorem. The repository README, blob f3cfcb92443c29a8ce87494f6c9a7b57e40129e9, was also read. Its complete-build and independent-checking claims are attributed to that source, not reproduced results of this work.

C.4 Leant source modules

`src/Leant/Synth/Verification.hs`, blob caee3c51cfd226e6c2bb9887f80293343abd486c, and `src/Leant/Synth/Length/Contract.hs`, blob a1f04001c6ccd6c5e018162fa4db7ff5b059972e, were read in full.

The first 200 source lines of `src/Leant/Synth/Engine.hs`, blob 1064c8215506e172401cd7c5c06d7e8222e0befa, and the first 220 lines of `src/Leant/Synth/Length/Handoff.hs`, blob 883e6fd2147fe9aa100c1691fb7fdb12b0f9380d, were inspected. They support the discussion of exported typed origins, explicit handoff contracts, retained provenance, and refusal conditions. This is not a whole-engine correctness audit or a solver execution test.

C.5 Companion validation and reproducibility

The archive contains the self-contained TeX source and built PDF, a build script, the finite Python companion, its actual test output and JSON receipt, and the uncompiled Lean core. The Python companion uses only the standard library. Its 24 passing test methods include several loops over small finite inputs; the method count is not a count of universal theorems or implemented language features.

The PDF build is checked for unresolved references and layout warnings and rendered for visual inspection. These document checks say nothing about mathematical kernel acceptance. The output manifest records hashes of the final distributable files; it does not claim to hash independently downloaded copies of the upstream repositories.

References

- [1] Codex. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals*. Round-two synthesis, revised 6 September 2026. Brainstorm repository, source and blob listed in Appendix C. <https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/synthesis/unified-report.tex>.
- [2] Claude. *Nine Workbenches: Certified Computation in a Mathematician’s Proof Language over Lean*. Round-two unified review, title dated 5 September 2026, revised version inspected 6 September 2026. https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/unified_report/unified_report.tex.
- [3] V. Reshetnikov and contributors. ProveIt, *ContinuousCDF.lean*. Source inspected at the registered revision. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/ContinuousCDF.lean>.
- [4] V. Reshetnikov and contributors. ProveIt, *AutonomousIteratedDeriv.lean*. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [5] Anthropic and credited contributors. *Fermat’s Last Theorem in Lean 4*, README and source attribution. <https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/README.md>.
- [6] Anthropic and credited contributors. *Def_AdicCompletionRingFunctoriality.lean*. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_AdicCompletionRingFunctoriality.lean.
- [7] Anthropic and credited contributors. *S_Rep_indBotSC_shortExact.lean*. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Rep_indBotSC_shortExact.lean.
- [8] V. Reshetnikov and contributors. Leant, *Leant.Synth.Verification*. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [9] V. Reshetnikov and contributors. Leant, *Leant.Synth.Engine*, header and exported interface inspected. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Engine.hs>.
- [10] V. Reshetnikov and contributors. Leant, *Leant.Synth.Length.Contract*. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Contract.hs>.
- [11] V. Reshetnikov and contributors. Leant, *Leant.Synth.Length.Handoff*, first 220 lines inspected. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Handoff.hs>.
- [12] Lean team. *Subtypes*. Lean Language Reference, latest documentation accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/>.
- [13] Lean team. *Axioms*. Lean Language Reference, including the discussion of nonparametric polymorphic functions. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Axioms/>.
- [14] Lean team. *Validating a Lean Proof*. Lean Language Reference. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [15] William Lovas and Frank Pfenning. *Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance*. 2010. arXiv:1009.1861. Abstract and publication metadata consulted; used here as a precedent, not as a technical dependency. <https://arxiv.org/abs/1009.1861>.
- [16] Jad Elkhaleq Ghalayini and Neel Krishnaswami. *Explicit Refinement Types*. 2023. arXiv:2311.13995. Abstract and publication metadata consulted. <https://arxiv.org/abs/2311.13995>.